

Universidade Federal do Rio de Janeiro

Instituto de Computação

Banco de Dados — Projeto de Data Warehouse

PARTE III · DOCUMENTO 1

Proposta Executiva

Solução Híbrida de Data Warehouse e Big Data

Operacionalização de uma frota conectada de locadoras em escala de cidade

Sistema de Locação de Veículos · Expansão da solução de Data Warehouse da Parte 2

COMPONENTES DO GRUPO

Tadeu Belfort Neto — DRE 119034813

Vicente Alves — DRE 1220044148

João Pedro de Lacerda — DRE 116076670

Rio de Janeiro

2026

1 Sumário Executivo

O consórcio de locadoras deseja evoluir de uma operação tradicional para uma **frota conectada**, capaz de ser acompanhada em tempo real, de reagir a emergências e de cobrar o cliente de forma adaptativa conforme o uso e o desgaste do veículo. Esta proposta, elaborada na posição de **consultoria de tecnologia**, recomenda e justifica a stack para cada etapa do pipeline de dados — e registra explicitamente o motivo de **não** adotar as alternativas concorrentes.

A solução recomendada é uma **plataforma híbrida que combina Data Warehouse (DW) e Big Data**: o DW dimensional já construído na Parte 2 (PostgreSQL) é preservado como fonte oficial do financeiro, e ao seu redor montamos um pipeline de Big Data — borda → ingestão → processamento → armazenamento → consumo — para o fluxo contínuo de telemetria da frota. O termo "híbrida" refere-se a essa convivência **DW + Big Data**; a infraestrutura recomendada é de **nuvem única**, não de nuvem híbrida.

RECOMENDAÇÃO DA CONSULTORIA

Adotar um **lakehouse unificado** (Databricks + Apache Spark + Delta Lake) sobre **nuvem única**, alimentado por **Apache Kafka gerenciado** e por **computação na borda**, com **armazenamento poliglota** (Redis · MongoDB · Delta · PostgreSQL) e consumo por **Grafana + BI (Business Intelligence) + ML (Machine Learning, aprendizado de máquina)**. O critério que orienta cada escolha é o **custo total de propriedade (TCO — Total Cost of Ownership)** com a stack mais simples e segura possível dentro do orçamento.

A frota gera um fluxo constante de dados que exige infraestrutura robusta de ingestão e armazenamento, mas o perfil do cliente — **baixa capacidade de investimento e necessidade de retorno rápido** — impõe disciplina de custo em todas as camadas. Por isso, **redução de custo é um critério de projeto explícito** desta proposta, e não apenas uma observação ao final.

2 Contexto e Problema de Negócio

O consórcio reúne locadoras que operam pátios em uma **mesma cidade** e querem automatizar e conectar a frota. Três frentes de valor sustentam o projeto: **operação em tempo real** (acompanhar a situação da frota e responder a emergências), **cobrança adaptativa** (precificar conforme uso e desgaste) e **redução de prejuízos** (menos custo com seguradoras e menos veículo parado em manutenção).

O perfil do cliente delimita a engenharia da solução:

- **Baixa capacidade de investimento e ROI (Return on Investment — retorno sobre o investimento) rápido** — o TCO é o critério que decide entre alternativas tecnicamente válidas.
- **Escala de cidade, não de megacidade** — pátios de uma mesma cidade; não há justificativa para superdimensionar a infraestrutura.
- **Fluxo contínuo de dados** — a frota conectada produz telemetria sem pausa, exigindo ingestão e armazenamento resilientes.
- **Reaproveitamento do DW da Parte 2** — o Data Warehouse dimensional já existe e deve ser estendido, não descartado.

Esse perfil explica o próprio formato do cliente: **locadoras menores** que, isoladamente, não comportariam uma plataforma de Big Data unem-se em **consórcio** para diluir o custo e operam na **escala de uma cidade** — não de uma megacidade. O enquadramento não é um detalhe de cenário: é o que sustenta, adiante, o **right-sizing** da solução (o descarte de Cassandra e de Flink) e a opção por **nuvem única**.

Mandato da consultoria: entregar a stack mais simples e segura possível dentro do orçamento — sem abrir mão da capacidade analítica nem da segurança da informação.

3 Objetivos da Solução

A solução precisa entregar resultado em duas dimensões complementares — a operacional e a financeira — sempre respeitando a limitação orçamentária, princípio que guia cada escolha desta proposta.

Operação em tempo real

- Acompanhamento contínuo da situação da frota
- Resposta imediata em emergências
- Mais clientes atendidos sem expandir a frota

Resultado financeiro

- Cobrança automática e adaptativa por desgaste
- Menos custo com seguradoras — prêmio e sinistro
- Menos prejuízo com veículo parado em manutenção

Vídeo como evidência: além de reduzir o prêmio do seguro, o registro audiovisual da emergência funciona como **prova do sinistro** — facilita a aceitação do reembolso pela seguradora e **agiliza a liquidação**, encurtando o tempo com o veículo parado.

4 Premissas e Enquadramento

O dimensionamento parte de premissas conservadoras, coerentes com a escala de cidade. Cada veículo ativo emite cerca de **1 mensagem por segundo** de telemetria de rotina, com **payload comprimido de 1–2 KB** (dados, não vídeo), durante uma janela ativa média de **~12 h/dia**. Os dados emergenciais — incluindo vídeo — são **event-driven e raros**, transmitidos apenas no gatilho. A tabela abaixo ancora três cenários de frota; o intermediário é tratado como baseline.

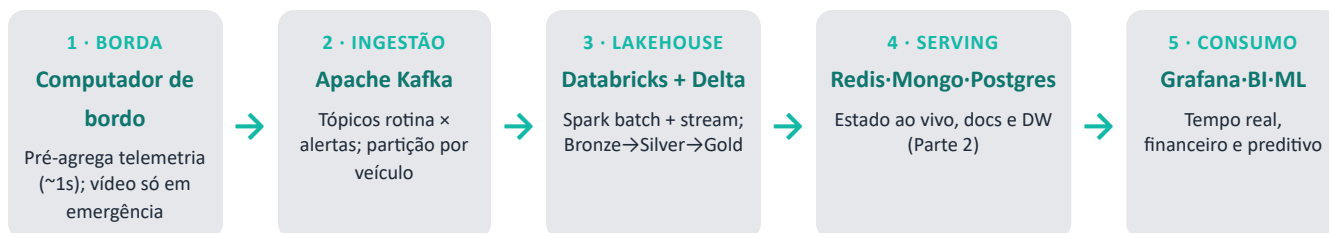
CENÁRIO (FROTA ATIVA)	PICO DE MENSAGENS	VAZÃO DE INGESTÃO	INGESTÃO BRUTA/DIA	INGESTÃO BRUTA/MÊS
1.000 veículos	~1.000 msg/s	~1,5 MB/s	~65 GB	~1,9 TB
2.000 veículos (baseline)	~2.000 msg/s	~3 MB/s	~130 GB	~3,9 TB
5.000 veículos	~5.000 msg/s	~7,5 MB/s	~325 GB	~9,7 TB

Volumes derivados das premissas (1 msg/s · 1–2 KB · ~12 h/dia). O vídeo emergencial entra à parte, por ser esporádico.

Esses números confirmam o ponto central: a telemetria de rotina, embora contínua, é **modesta em volume** quando comprimida na borda. É o vídeo emergencial — caro e volumoso — que precisa ser contido por desenho, transmitido apenas quando há um evento que o justifique. Essa distinção orienta toda a arquitetura.

5 Arquitetura de Referência

O pipeline percorre cinco camadas, do carro ao dashboard. A decisão estrutural mais importante é a **separação entre o caminho quente (emergências) e o caminho frio (telemetria de rotina) já na borda**: cada tipo de dado segue por um trajeto dimensionado para o seu custo e para a sua urgência.



Sustentando as cinco camadas, três pilares transversais: **nuvem única** (stack gerenciada, modelo opex, sem on-premise), **segurança** (criptografia, IAM — Identity and Access Management, gestão de identidade e acesso —, VPC — Virtual Private Cloud, rede virtual privada — e LGPD — Lei Geral de Proteção de Dados — em região brasileira) e **orquestração** (Databricks Workflows e governança de dados). As seções seguintes detalham cada camada e justificam as escolhas.

6 Camada 1 — Coleta na Borda (Edge Computing)

O computador de bordo de cada veículo é a primeira — e mais importante — alavanca de custo da solução. Em vez de transmitir tudo em tempo integral, ele **pré-processa os dados localmente** e decide o que sobe para a nuvem, separando desde a origem a rotina da emergência.

Dados de rotina · caminho frio

- Telemetria compactada a cada ~1 s
- Posição e situação geral do veículo
- Base da cobrança por desgaste
- Transmissão barata e espaçada

Dados emergenciais · caminho quente

- Vídeo/imagens apenas no gatilho (caro)
- (a) Problema interno → alerta + snapshot
- (b) Risco externo → imagens prévias ou streaming
- (c) Botão do motorista → streaming contínuo

Os três gatilhos de emergência cobrem situações distintas, cada um com uma resposta proporcional ao risco:

- **Problema interno do veículo** — alerta imediato com **snapshot** do estado no instante da detecção. Se uma nova ocorrência surgir dentro de uma janela de tempo, são enviadas também as **imagens de antes e depois** (recomendado: 5 min de cada lado). Quando a falha inviabiliza o uso do carro (ex.: enguiço), o evento **escala para prioridade máxima**, pois implica retirar o veículo da frota ativa.
- **Risco externo** — desdobra-se em dois níveis. Numa **desaceleração brusca**, sobem apenas as imagens que **antecederam** o evento (recomendado: último 1 min), sem streaming. Já o acionamento dos **sensores de pré-impacto** (risco maior) dispara um **vídeo mais longo** (mín. 5 min) e **streaming ao vivo** por igual duração.
- **Botão de emergência do motorista** — nível máximo de prioridade: alerta, dados do **último 1 min** e **streaming contínuo** até a **desativação manual**, pelo próprio motorista ou pela central.

Em todos os casos, o vídeo só existe quando há motivo para existir — é o que mantém o custo de banda sob controle.

PRINCIPAL ALAVANCA DE CUSTO

Pré-processar a bordo elimina o gasto recorrente mais caro de toda a operação — a **banda móvel**. Transmitir vídeo continuamente de milhares de veículos seria proibitivo; restringi-lo ao gatilho de emergência é o que torna a solução economicamente viável na escala de cidade.

7 Camada 2 — Ingestão de Eventos (Apache Kafka)

A ingestão usa um **barramento de eventos gerenciado** (Apache Kafka, via Amazon MSK — Managed Streaming for Apache Kafka — ou Confluent Cloud) que desacopla a frota dos consumidores e absorve picos sem perder dados. O Kafka é o ponto de entrada de tudo o que sai dos veículos.

- **Tópicos separados** — telemetria de rotina e alertas de emergência trafegam em tópicos distintos, de modo que um alerta crítico nunca fique na fila atrás da telemetria de baixa prioridade.

- **Partição por `vehicle_id`** — garante a ordenação estrita dos eventos de cada veículo (por timestamp) mesmo sob carga, o que é essencial para reconstruir corretamente uma viagem ou um incidente.
- **Tolerância a falhas e replay** — as mensagens ficam retidas por um período configurável (recomendado: **7 dias** para a telemetria de rotina e **30 dias** para os alertas), com replicação entre brokers. Esse histórico permite **reprocessar** os dados a qualquer momento sem precisar de uma fonte adicional — é a base do modelo de reprocessamento adotado na Camada 3.
- **Consumidores independentes (consumer groups)** — o mesmo tópico é lido em paralelo por vários consumidores sem interferência mútua: o Spark (analítico), o serviço de alertas de emergência e o arquivamento no lakehouse leem o mesmo fluxo, cada um no seu ritmo.
- **Serialização compacta** — mensagens em formato binário (Avro ou Protobuf) com **Schema Registry**: menos bytes na rede móvel (custo) e um contrato de dados validado entre o veículo e a nuvem (qualidade).
- **Serviço gerenciado** — zero servidor Kafka para a equipe operar, coerente com a ausência de time de infraestrutura dedicado.

Por que não enviar requisições REST (Representational State Transfer, o padrão de APIs sobre HTTP) direto ao banco: sob o pico de milhares de mensagens por segundo, o acesso direto ao banco vira gargalo de concorrência e leva à perda de eventos. O Kafka existe justamente para amortecer esse pico.

8 Camada 3 — Processamento (Lakehouse Unificado)

O coração analítico da solução é um **lakehouse**: um único motor de processamento sobre uma única camada de armazenamento transacional, eliminando a proliferação de tecnologias que historicamente encarece e complica pipelines de Big Data.

- **Databricks + Spark Structured Streaming** — um só motor trata **batch e streaming com o mesmo código**, atendendo tanto a análise contínua quanto o caminho quente das emergências.
- **Delta Lake em arquitetura medalhão** — os dados evoluem de **Bronze** (bruto) para **Silver** (limpo e anonimizado) e **Gold** (pronto para consumo), com garantias **ACID** (Atomicidade, Consistência, Isolamento e Durabilidade) válidas entre batch e stream.
- **Modelo Kappa** — o reprocessamento histórico é feito por **replay** a partir do Kafka, evitando a duplicação de lógica que caracteriza a arquitetura Lambda (um código para batch e outro para stream).

Comparação formal: Lambda × Kappa

A escolha do Kappa sobre Delta Lake resolve o impasse clássico entre as duas arquiteturas de referência. A **Lambda** mantém dois caminhos independentes — a *batch layer* (visão completa e correta, recalculada periodicamente) e a *speed layer* (visão aproximada e imediata) — reconciliados numa camada de serving. É robusta, porém obriga a manter **duas bases de código sincronizadas** para a mesma regra de negócio. A **Kappa** elimina a batch layer: tudo é tratado como fluxo, e o reprocessamento histórico é um **replay** do próprio Kafka. O Delta Lake completa o desenho ao dar ao stream único a robustez transacional que era o argumento da batch layer.

CRITÉRIO	LAMBDA	KAPPA (ADOTADA)
Caminhos de processamento	Dois: batch layer + speed layer	Um: stream; o "batch" é replay do Kafka
Base de código	Duplicada — mesma regra escrita duas vezes	Única — mesmo código Spark para batch e stream
Consistência batch × stream	Reconciliação manual na camada de serving	Transações ACID do Delta Lake para os dois
Custo operacional	Dois pipelines e dois clusters para manter	Um pipeline, um cluster elástico
Quando faz sentido	Reprocessos massivos com lógicas realmente distintas	Fluxo contínuo com reprocesso ocasional — o nosso caso

Arquitetura medalhão aplicada ao projeto

- **Bronze (bruto)** — os eventos exatamente como chegaram do Kafka, imutáveis; servem de trilha de auditoria e de fonte para reprocessamento.
- **Silver (curado)** — telemetria **deduplicada, validada e anonimizada** (atendendo à LGPD), enriquecida com os cadastros de veículo, locação e cliente vindos do DW.
- **Gold (consumo)** — agregados de negócio prontos: quilometragem e tempo de uso por locação, indicadores de desgaste por veículo, séries de demanda por região — o que alimenta a cobrança adaptativa, o BI e o ML.

O que roda em streaming e o que roda em batch

Streaming · contínuo, 24/7

- Detecção e triagem de emergências (caminho quente)
- Atualização do estado ao vivo da frota no Redis
- Janelas temporais (ex.: médias por 1–5 min) para congestionamento e anomalias
- Aterrissagem contínua dos eventos na camada Bronze

Batch · agendado, em instâncias spot

- Consolidação diária de quilometragem e tempo de uso
- Apuração da cobrança adaptativa pós-devolução
- Treinamento dos modelos de previsão de falhas e demanda
- Compactação e downsampling do histórico (custo de storage)

Nos dois modos o motor é o mesmo — o Spark Structured Streaming processa o fluxo em **micro-batches**, com latência de poucos segundos, suficiente para o alerta de emergência chegar à central em tempo hábil. É essa unificação que dispensa um segundo motor de streaming.

Mantido (right-sizing)

- Spark (batch + stream) sobre Delta Lake — um motor, um código

Descartado (right-sizing)

- Hadoop / MapReduce — paradigma legado: grava resultado intermediário em disco a cada etapa, enquanto o Spark processa em memória (ordens de grandeza mais rápido) com API de mais alto nível
- Apache Flink — especializado em latência sub-segundo evento a evento, que não se justifica na escala de cidade; o Spark Structured Streaming cobre o caminho quente com segundos de latência e um motor a menos para operar

O princípio aqui é **right-sizing**: adotar a peça certa para a escala do problema. Menos peças significa menos código, menos a operar e menos custo — sem perda de capacidade analítica.

9 Camada 4 — Armazenamento Poliglota

Não existe um banco único ideal para todos os perfis de dado da solução. Adotamos **persistência poliglota**, combinando bancos **NoSQL** ("not only SQL" — modelos não relacionais como chave-valor e documentos) com o relacional: cada tipo de dado vai para a tecnologia que melhor o atende em custo e desempenho.

TECNOLOGIA	PAPEL	POR QUE
Redis	Estado ao vivo da frota + cache de rotas	Leitura sub-milissegundo para o painel em tempo real
MongoDB Atlas	Documentos: viagens, sinistros, inspeções	Schema flexível para dados semiestruturados; gerenciado
Delta Lake	Histórico analítico de telemetria	Barato, escalável e com ACID; é a camada do lakehouse
PostgreSQL (DW)	Financeiro e cobrança — reúso da Parte 2	ACID estrito; zero licença nova; modelo dimensional pronto

As quatro peças formam um **ciclo de vida do dado** alinhado ao custo: o evento chega pelo Kafka e atualiza o **Redis** (estado ao vivo, com expiração automática — só o presente importa ali); o mesmo evento aterrissa no **Delta Lake**, onde vive o histórico completo em storage barato; documentos operacionais (dossiê de viagem, sinistro, inspeção) consolidam-se no **MongoDB**; e os agregados financeiros apurados no lakehouse são gravados no **PostgreSQL/DW**, que permanece a fonte oficial de cobrança e relatório. Cada dado é pago pelo que ele vale em cada fase da sua vida — quente, morno ou frio.

Por que descartamos o Cassandra: ele foi projetado para escrita massiva distribuída em múltiplos data centers — um perfil de megacidade. Na escala de cidade, seria **superdimensionar**. O Delta Lake cobre o histórico de telemetria com menos custo operacional e ainda integra nativamente ao motor de processamento.

10 Camada 5 — Consumo (Analytics, Dashboards e IA)

O valor dos dados se realiza no consumo. Três frentes atendem públicos e perguntas diferentes, todas sobre uma base de dados única e consistente.

Tempo real · Grafana

- Frota: ativos, circulação, manutenção
- Viagens em andamento e emergências
- Alimentado por Redis / streaming

Negócio · BI sobre o DW

- Reservas, locações, cobranças, financeiro
- Metabase (open-source) ou Power BI
- Cobrança adaptativa pós-devolução

Inteligência · ML

- Previsão de falhas mecânicas
- Previsão de demanda por região
- Databricks ML / MLflow

Em conjunto, as três frentes cobrem os **quatro tipos de análise** — descritiva (o que aconteceu), diagnóstica (por que aconteceu), preditiva (o que vai acontecer) e prescritiva (o que fazer) — sobre uma base única. O Grafana atende ao monitoramento ao vivo do caminho quente; o BI corporativo atende à visão histórica e financeira sobre o DW; e o ML antecipa falhas e demanda, fechando o ciclo de valor.

Roadmap — otimização de rotas e frota não tripulada

A telemetria de rotina, além de embasar a cobrança, habilita uma evolução natural já prevista na especificação original: usar o histórico de deslocamento para **otimizar rotas** e, no limite, conduzir **veículos não tripulados**. O **ML preditivo** (Camada 5) aprende os padrões de trânsito e de demanda, e o **cache de rotas no Redis** (Camada 4) entrega a rota escolhida ao veículo em tempo real. É a **mesma infraestrutura, sem novas peças**, elevando a **eficiência da frota**: menos quilômetro ocioso, melhor posicionamento dos carros e, adiante, a condução autônoma apoiada nos dados que a solução já coleta.

11 Nuvem e Segurança

A solução roda em **nuvem única**, sem componente on-premise nem nuvem híbrida. Essa escolha foi reaberta e confirmada à luz da experiência de mercado: o modelo híbrido adicionaria custo de equipamento e de pessoal e elevaria muito a complexidade de sincronização entre ambientes — sem benefício proporcional na escala de cidade.

Por que nuvem única (sem híbrido)

- Opex, não capex — sem comprar hardware
- Sem equipe de data center para manter
- Elimina a sincronização on-prem ↔ cloud
- Stack portátil entre provedores

Como mantemos a segurança

- Criptografia em repouso e em trânsito
- IAM de menor privilégio + isolamento por VPC
- LGPD: região brasileira + anonimização na Silver
- Auditoria e políticas de retenção

Sigilo de dados **não exige servidor próprio**: controles cloud-native (criptografia, IAM, VPC) somados à **residência de dados no Brasil** atendem aos requisitos de segurança e de LGPD com menor custo e menor complexidade.

12 Estratégia de Redução de Custos

Por se tratar de um cliente com orçamento restrito e necessidade de ROI rápido, a redução de custo é o **fio condutor** da arquitetura — materializada em seis alavancas que atravessam todas as camadas.

ALAVANCA	COMO REDUZ CUSTO
Edge corta a banda	Só a emergência sobe vídeo; elimina o maior custo recorrente da operação
Open-source primeiro	PostgreSQL, Kafka, Spark e Grafana — zero custo de licença
Batch em instâncias spot	Só o caminho quente fica 24/7; o batch e o ML rodam de forma pontual e barata
Armazenamento em camadas	Hot (Redis) → warm (MongoDB) → cold (Delta) com TTL (Time to Live — expiração automática) e downsampling
Reúso do DW da Parte 2	Sem reescrever nem relicenciar o que já existe e funciona
Right-sizing como princípio	Infraestrutura de cidade, não de megacidade — nada superdimensionado

Há ainda um retorno que compensa o único fluxo caro que mantemos. O **vídeo emergencial** não é só custo: é a **evidência do sinistro** que facilita a aceitação do reembolso e **agiliza a liquidação** junto à seguradora. O gasto pontual de banda no momento da emergência converte-se em **recuperação financeira** e em menos dias de veículo parado — um custo que, na prática, se paga.

A comparação entre **nuvem e on-premise** reforça a escolha. O on-premise embute capex de hardware, energia, refrigeração, redundância e — o maior custo escondido — **pessoal de operação**. A nuvem converte tudo isso em opex previsível e elástico, dispensando o time de data center. Sob a premissa de orçamento apertado e ROI rápido, a **nuvem única vence em TCO**.

Resultado: TCO baixo e ROI rápido — exatamente o que o perfil do cliente exige, sem sacrificar segurança nem capacidade analítica.

13 Extra — Concierge de Viagem por Voz

Como diferencial — sugerido como extra ao escopo — propomos, em nível de **especificação funcional**, um assistente de IA embarcado que melhora a experiência da viagem: recomenda lugares, ajusta a rota e conecta-se à agenda do cliente, por voz. A peça-chave do desenho é **onde ele roda**: no **mesmo computador de bordo** que já pré-processa a telemetria e detecta as emergências (Camada 1). Não há hardware novo — reaproveita-se um ativo que o cliente já terá instalado em cada veículo.

- **Hospedado na borda** — o computador de bordo já previsto passa a abrigar também a interface de voz e a chamada ao LLM+RAG; o custo é **marginal**, sem novo equipamento.
- **Voz → texto → voz** — reconhecimento de fala (STT — Speech-to-Text) e síntese de voz (TTS — Text-to-Speech) para uma interação natural ao volante.
- **LLM + RAG** — modelo de linguagem de grande porte (LLM — Large Language Model) consultando, via geração aumentada por recuperação (RAG — Retrieval-Augmented Generation), uma base local de pontos de interesse (POIs — Points of Interest) da cidade, com respostas ancoradas em dados reais.
- **Ação no veículo** — reprograma a rota no GPS (Global Positioning System) via API (Application Programming Interface — interface de programação), fechando o ciclo entre recomendação e ação.
- **Desacoplado do pipeline** — opera como componente opcional e independente; **não infla** o fluxo central de dados (borda → Kafka → lakehouse).

Por rodar sobre um ativo já instalado e seguir **desacoplado** do núcleo de dados, o concierge entrega alto impacto na percepção de valor com **esforço e custo marginal contidos** — permanecendo como especificação funcional, não implementação.

14 Decisões-Chave — Síntese

A tabela consolida as escolhas da consultoria em cada ponto do pipeline, com a alternativa descartada e a justificativa — o **porquê de não** que o exercício pede.

DECISÃO	ESCOLHIDO	DESCARTADO	POR QUÊ
Arquitetura de dados	Kappa sobre Delta Lake	Lambda	Evita duplicar a lógica batch/stream; Delta dá ACID às duas
Processamento	Spark (Databricks)	Hadoop/MapReduce; Flink	Legado e latência sub-segundo desnecessária na escala de cidade
Plataforma	Databricks + Delta	Dataproc avulso	Lakehouse nativo: consolida processamento, storage e ML
Telemetria histórica	Delta Lake	Cassandra	Cassandra superdimensiona para escala de cidade
Infraestrutura	Nuvem única	Híbrido / multi-cloud	Sem capex nem custo de sync; opex elástico
Dashboard	Grafana + BI	Só um deles	Papéis distintos: tempo real (Grafana) e financeiro (BI/DW)
Concierge de voz	Incluído (spec)	Deixar de fora	Alto retorno de imagem, baixo esforço por ser desenho funcional

15 Conclusão

Esta proposta fecha um ciclo de três etapas. Na Parte 1, o consórcio ganhou um banco transacional (OLTP — Online Transaction Processing) que sustenta a operação diária de reservas, locações e cobranças. Na Parte 2, esses dados foram consolidados em um Data Warehouse dimensional, que unificou as locadoras em uma visão analítica única do negócio. A Parte 3 estende essa fundação para o mundo do **Big Data**: a frota conectada passa a produzir dados continuamente, e a plataforma aqui proposta — borda, Kafka, lakehouse, armazenamento poliglota e consumo — transforma esse fluxo em operação em tempo real, cobrança adaptativa e redução de prejuízo, **sem descartar nada do que foi construído antes**.

A recomendação se sustenta em cinco atributos que respondem diretamente ao mandato do cliente:

- **Simples** — um lakehouse no lugar de três motores e duas camadas; menos peças a operar.
- **Seguro** — criptografia, IAM, VPC e LGPD em região brasileira, sem necessidade de servidor próprio.
- **Econômico** — nuvem única, stack open-source, edge cortando banda e reuso do DW da Parte 2.
- **Completo** — tempo real, histórico, cobrança adaptativa, ML e o concierge de voz.
- **Bem dimensionado** — a escala de cidade é tratada como virtude de projeto, não como limitação.

Implantação em fases

Coerente com a restrição de orçamento e a exigência de ROI rápido, a implantação é **incremental** — cada fase entrega valor mensurável e valida o custo antes de liberar a seguinte:

- **Fase 1 · Piloto** — subconjunto da frota com o pipeline mínimo (borda → Kafka → Spark/Delta → Grafana): valida os gatilhos de emergência, o custo real de banda e a operação do painel ao vivo.
- **Fase 2 · Frota completa** — expansão da telemetria a todos os veículos, cobrança adaptativa integrada ao DW e BI financeiro em produção — é aqui que o retorno financeiro do projeto se materializa.
- **Fase 3 · Inteligência** — ML preditivo (falhas e demanda), otimização de rotas e o concierge de voz como diferencial de experiência — valor incremental sobre a mesma infraestrutura, sem novas peças.

O resultado esperado responde, ponto a ponto, às três frentes de valor do negócio: a central **enxerga e socorre** a frota em tempo real; a cobrança passa a refletir o **uso e o desgaste reais** de cada locação; e o histórico analítico reduz prejuízo com seguradora, manutenção e veículo parado. Tudo isso com TCO contido por desenho — a economia não é um ajuste posterior, é o critério que escolheu cada peça desta arquitetura.

"Do dado operacional ao Big Data em tempo real — uma plataforma única, enxuta e segura para decidir e operar a frota."

16 Glossário de Siglas e Termos

Siglas

SIGLA	SIGNIFICADO
ACID	Atomicidade, Consistência, Isolamento e Durabilidade — garantias de integridade de uma transação de banco de dados.
API	Application Programming Interface — interface de programação que permite a integração entre sistemas.
BI	Business Intelligence — ferramentas de análise e relatórios sobre dados do negócio.
DW	Data Warehouse — repositório analítico que consolida dados de várias fontes em modelo dimensional.
GPS	Global Positioning System — sistema de posicionamento por satélite.
IAM	Identity and Access Management — gestão de identidade e controle de acesso na nuvem.
LGPD	Lei Geral de Proteção de Dados (Lei nº 13.709/2018) — legislação brasileira de privacidade.
LLM	Large Language Model — modelo de linguagem de grande porte (base dos assistentes de IA).
ML	Machine Learning — aprendizado de máquina; modelos que aprendem padrões a partir de dados.
MSK	Amazon Managed Streaming for Apache Kafka — serviço de Kafka gerenciado da AWS.
NoSQL	"Not only SQL" — bancos de dados não relacionais (chave-valor, documentos, colunar, grafos).
OLTP	Online Transaction Processing — banco transacional que sustenta a operação do dia a dia.
POI	Point of Interest — ponto de interesse (restaurantes, museus etc.) em uma base geográfica.
RAG	Retrieval-Augmented Generation — técnica que ancora as respostas de um LLM em uma base de dados real.
REST	Representational State Transfer — padrão de comunicação de APIs sobre HTTP.
ROI	Return on Investment — retorno sobre o investimento.
STT / TTS	Speech-to-Text (reconhecimento de fala) / Text-to-Speech (síntese de voz).
TCO	Total Cost of Ownership — custo total de propriedade (aquisição + operação + manutenção).
TTL	Time to Live — tempo de vida de um dado, após o qual ele expira automaticamente.
VPC	Virtual Private Cloud — rede virtual privada e isolada dentro da nuvem pública.

Termos técnicos

TERMO	DEFINIÇÃO
Arquitetura Kappa / Lambda	Padrões de referência para Big Data: a Lambda mantém caminhos separados de batch e stream; a Kappa trata tudo como fluxo único (Seção 8).
Arquitetura medalhão	Organização do lakehouse em camadas Bronze (bruto) → Silver (curado) → Gold (pronto para consumo).
Batch / Streaming	Processamento em lote (agendado, sobre dados acumulados) / processamento contínuo do fluxo à medida que os dados chegam.
Caminho quente / frio	Trajetos separados para dados urgentes (emergências) e dados de rotina (telemetria), cada um dimensionado ao seu custo.
Capex / Opex	Despesa de capital (compra de ativos, ex.: servidores) / despesa operacional (pagamento pelo uso, ex.: nuvem).
Downsampling	Redução da resolução do histórico antigo (ex.: de 1 leitura/s para 1/min) para economizar armazenamento.
Edge computing	Computação na borda — processar os dados no próprio dispositivo (o computador de bordo) antes de enviá-los à nuvem.
Instâncias spot	Capacidade ociosa da nuvem vendida com grande desconto, ideal para cargas interrompíveis como o batch.
Lakehouse	Arquitetura que une o custo baixo do data lake às garantias transacionais do data warehouse (ex.: Delta Lake).
Payload	Conteúdo útil de uma mensagem transmitida pela rede.
Replay	Releitura do histórico retido no Kafka para reprocessar dados sem uma fonte adicional.
Right-sizing	Dimensionar cada componente para a escala real do problema — nem falta, nem excesso.
Telemetria	Medição remota: os dados de sensores que o veículo transmite (posição, velocidade, estado do motor etc.).

Repositório do projeto: github.com/tadeupires21-sketch/locadora-db